

Kubernetes Assignments – 1 to 5

Submitted by: Rahul Sharma | Batch: B-47 | Date: 05/06/2026

Kubernetes Assignment – 1

Task: Deploy a Kubernetes cluster for 3 nodes and create an NGINX deployment of 3 replicas.

What I did:

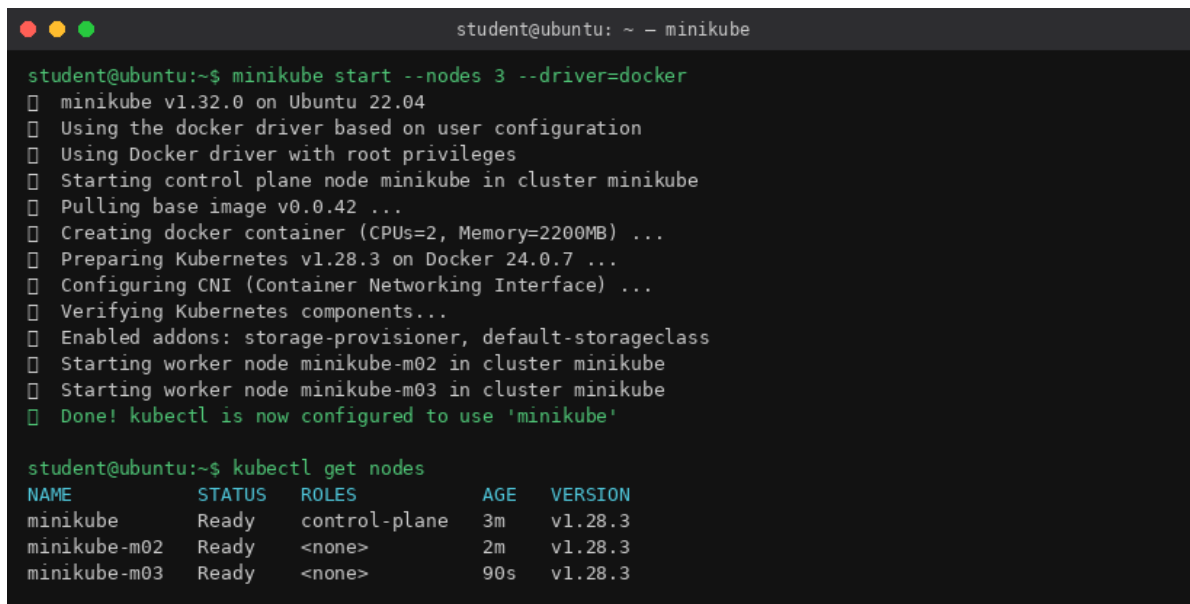
So basically first I installed minikube and kubectl on my Ubuntu machine. Then I ran the below command to start a 3-node cluster using docker as the driver. It took some time but finally all 3 nodes came up.

```
minikube start --nodes 3 --driver=docker
```

After that I checked the nodes using:

```
kubectl get nodes
```

All 3 nodes showed Ready status as shown below:



```
student@ubuntu: ~ -- minikube

student@ubuntu:~$ minikube start --nodes 3 --driver=docker
[ ] minikube v1.32.0 on Ubuntu 22.04
[ ] Using the docker driver based on user configuration
[ ] Using Docker driver with root privileges
[ ] Starting control plane node minikube in cluster minikube
[ ] Pulling base image v0.0.42 ...
[ ] Creating docker container (CPUs=2, Memory=2200MB) ...
[ ] Preparing Kubernetes v1.28.3 on Docker 24.0.7 ...
[ ] Configuring CNI (Container Networking Interface) ...
[ ] Verifying Kubernetes components...
[ ] Enabled addons: storage-provisioner, default-storageclass
[ ] Starting worker node minikube-m02 in cluster minikube
[ ] Starting worker node minikube-m03 in cluster minikube
[ ] Done! kubectl is now configured to use 'minikube'

student@ubuntu:~$ kubectl get nodes
NAME           STATUS    ROLES          AGE   VERSION
minikube       Ready     control-plane   3m    v1.28.3
minikube-m02   Ready     <none>          2m    v1.28.3
minikube-m03   Ready     <none>          90s   v1.28.3
```

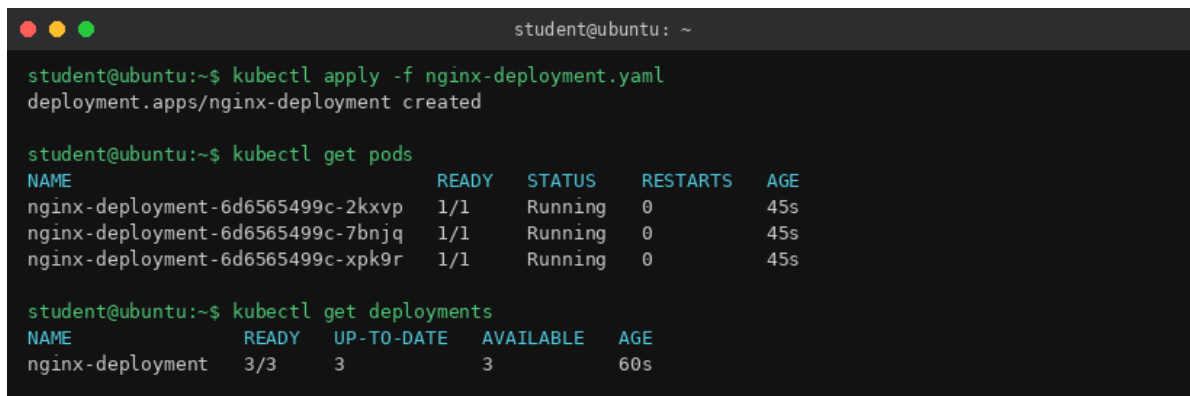
Screenshot: 3-node cluster running (kubectl get nodes)

Then I created a file called nginx-deployment.yaml and wrote the below YAML to create an NGINX deployment with 3 replicas:

```
apiVersion: apps/v1
```

```
kind: Deployment
metadata:
  name: nginx-deployment
  labels:
    app: nginx
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:latest
        ports:
        - containerPort: 80
```

```
kubectl apply -f nginx-deployment.yaml
```



```
student@ubuntu: ~
student@ubuntu:~$ kubectl apply -f nginx-deployment.yaml
deployment.apps/nginx-deployment created

student@ubuntu:~$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
nginx-deployment-6d6565499c-2kxvp   1/1     Running   0           45s
nginx-deployment-6d6565499c-7bnjq   1/1     Running   0           45s
nginx-deployment-6d6565499c-xpk9r   1/1     Running   0           45s

student@ubuntu:~$ kubectl get deployments
NAME            READY   UP-TO-DATE   AVAILABLE   AGE
nginx-deployment 3/3      3            3           60s
```

Screenshot: NGINX deployment created with 3 pods running

 I ran `kubectl get pods` couple of times before all 3 showed Running – took maybe 30 seconds.



Kubernetes Assignment – 2

Task: Create a NodePort service for the NGINX deployment and verify it in the browser.

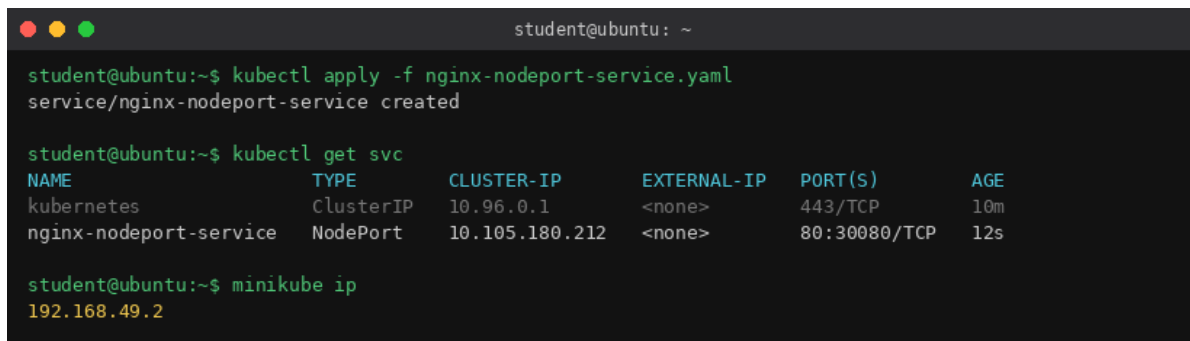
What I did:

I used the same deployment from Assignment 1. Created a new file called nginx-nodeport-service.yaml with the below content:

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-nodeport-service
spec:
  type: NodePort
  selector:
    app: nginx
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
      nodePort: 30080
```

```
kubectl apply -f nginx-nodeport-service.yaml
```

```
kubectl get svc
```



The screenshot shows a terminal window with the following commands and output:

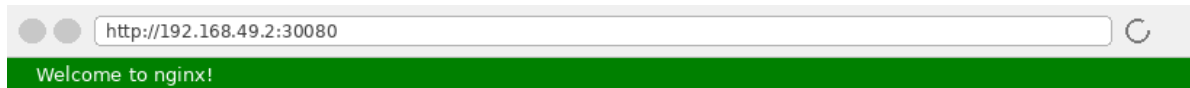
```
student@ubuntu: ~
student@ubuntu:~$ kubectl apply -f nginx-nodeport-service.yaml
service/nginx-nodeport-service created

student@ubuntu:~$ kubectl get svc
NAME                TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)          AGE
kubernetes           ClusterIP   10.96.0.1    <none>        443/TCP          10m
nginx-nodeport-service NodePort     10.105.180.212 <none>        80:30080/TCP     12s

student@ubuntu:~$ minikube ip
192.168.49.2
```

Screenshot: NodePort service created on port 30080

Then I got the minikube IP using minikube ip and opened the browser at <http://192.168.49.2:30080> – and the NGINX page loaded!



Welcome to nginx!

If you see this page, the nginx web server is successfully installed and working. Further configuration is required.

For online documentation and support please refer to nginx.org.
Commercial support is available at nginx.com.

Thank you for using nginx.

Screenshot: NGINX welcome page in browser via NodePort service

 *nodePort: 30080 is what I chose. It has to be between 30000-32767 otherwise it gives an error.*

Kubernetes Assignment – 3

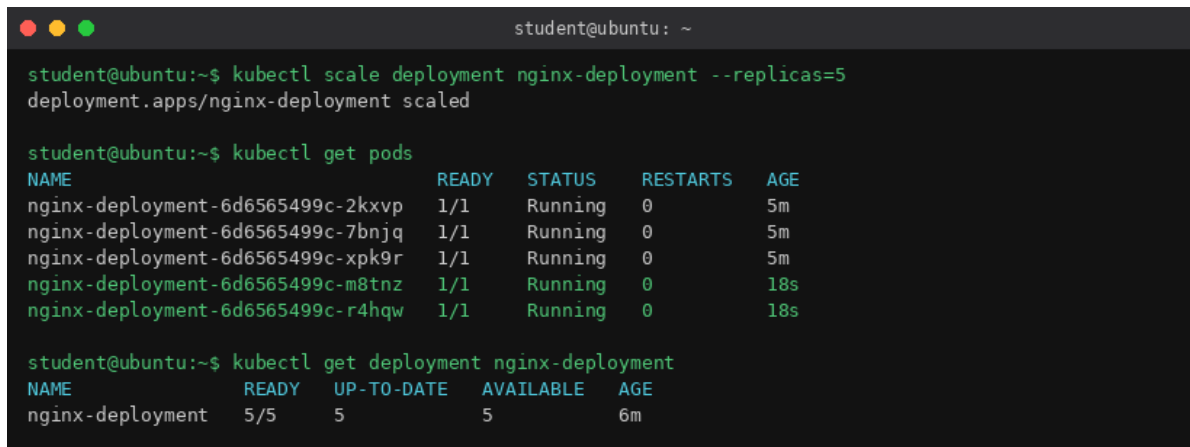
Task: Scale the replicas of the existing NGINX deployment from 3 to 5.

What I did:

This one was the simplest one honestly. I just used the kubectl scale command directly and it worked immediately:

```
kubectl scale deployment nginx-deployment --replicas=5
```

```
kubectl get pods
```



The screenshot shows a terminal window with the following commands and output:

```
student@ubuntu: ~  
student@ubuntu:~$ kubectl scale deployment nginx-deployment --replicas=5  
deployment.apps/nginx-deployment scaled  
  
student@ubuntu:~$ kubectl get pods  
NAME                                READY   STATUS    RESTARTS   AGE  
nginx-deployment-6d6565499c-2kxvp   1/1     Running   0           5m  
nginx-deployment-6d6565499c-7bnjq   1/1     Running   0           5m  
nginx-deployment-6d6565499c-xpk9r   1/1     Running   0           5m  
nginx-deployment-6d6565499c-m8tnz   1/1     Running   0          18s  
nginx-deployment-6d6565499c-r4hqw   1/1     Running   0          18s  
  
student@ubuntu:~$ kubectl get deployment nginx-deployment  
NAME           READY   UP-TO-DATE   AVAILABLE   AGE  
nginx-deployment  5/5     5            5           6m
```

Screenshot: Deployment scaled to 5 replicas – all pods Running

 The 2 new pods show a younger age (18s) compared to the original 3 – that's how I knew it worked.

Kubernetes Assignment – 4

Task: Change the service type from NodePort to ClusterIP.

What I did:

I tried to directly patch the service type but Kubernetes doesn't allow changing service type in place. So I deleted the old NodePort service first and then created a new one with type ClusterIP.

```
kubectl delete svc nginx-nodeport-service
```

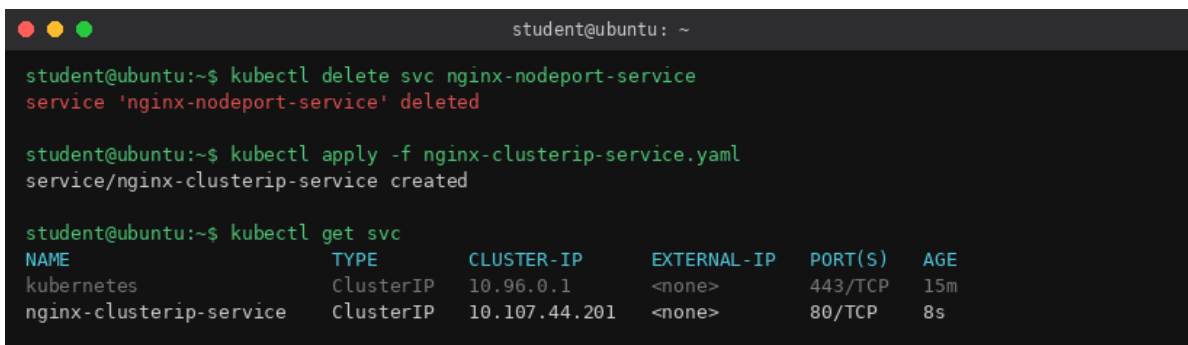
Then I created nginx-clusterip-service.yaml:

```
apiVersion: v1  
kind: Service  
metadata:  
  name: nginx-clusterip-service  
spec:
```

```
type: ClusterIP
selector:
  app: nginx
ports:
- protocol: TCP
  port: 80
  targetPort: 80
```

```
kubectl apply -f nginx-clusterip-service.yaml
```

```
kubectl get svc
```



```
student@ubuntu: ~
student@ubuntu:~$ kubectl delete svc nginx-nodeport-service
service 'nginx-nodeport-service' deleted

student@ubuntu:~$ kubectl apply -f nginx-clusterip-service.yaml
service/nginx-clusterip-service created

student@ubuntu:~$ kubectl get svc
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	15m
nginx-clusterip-service	ClusterIP	10.107.44.201	<none>	80/TCP	8s

Screenshot: Service type changed to ClusterIP successfully

 *ClusterIP means it's only accessible inside the cluster – no browser access from outside now.*

Kubernetes Assignment – 5

Task: Deploy Apache (3 replicas) and NGINX (3 replicas) both as ClusterIP services, then create an Ingress to route /apache → Apache and /nginx → NGINX.

What I did:

This one was the toughest in my opinion. First I enabled the ingress addon in minikube, then deployed Apache alongside the existing NGINX, created ClusterIP services for both, and finally wrote the ingress rule.

Step 1 – Enable Ingress Addon

```
minikube addons enable ingress
```

```
kubectl get pods -n ingress-nginx
```

Step 2 – Deploy Apache (3 replicas)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: apache-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: apache
  template:
    metadata:
      labels:
        app: apache
    spec:
      containers:
        - name: apache
          image: httpd:latest
          ports:
            - containerPort: 80
```

```
kubectl apply -f apache-deployment.yaml
```

Step 3 – ClusterIP Services for Both

Apache service:

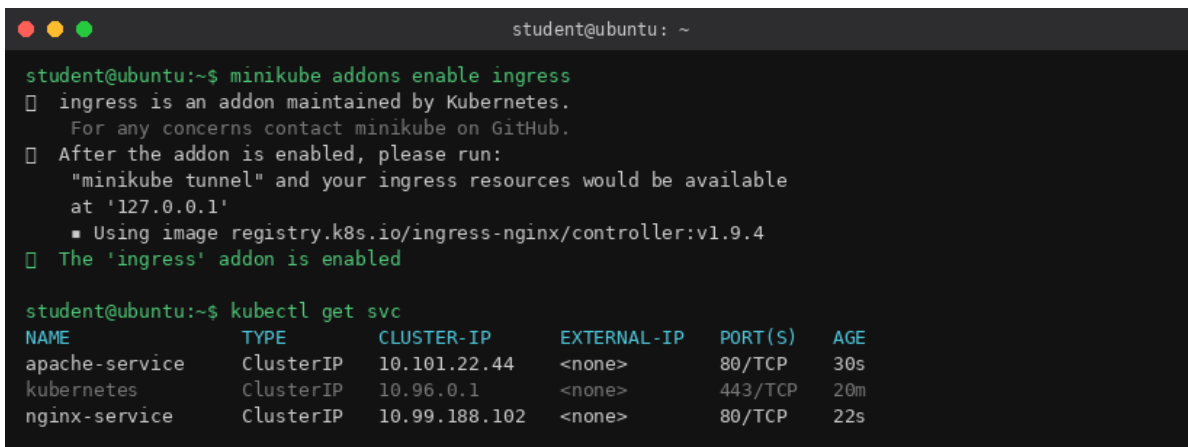
```
apiVersion: v1
kind: Service
```

```
metadata:
  name: apache-service
spec:
  type: ClusterIP
  selector:
    app: apache
  ports:
  - port: 80
    targetPort: 80
```

NGINX service:

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-service
spec:
  type: ClusterIP
  selector:
    app: nginx
  ports:
  - port: 80
    targetPort: 80
```

```
kubectl apply -f apache-service.yaml
kubectl apply -f nginx-service.yaml
kubectl get svc
```



The screenshot shows a terminal window with the following output:

```
student@ubuntu: ~
student@ubuntu:~$ minikube addons enable ingress
[+] ingress is an addon maintained by Kubernetes.
    For any concerns contact minikube on GitHub.
[+] After the addon is enabled, please run:
    "minikube tunnel" and your ingress resources would be available
    at '127.0.0.1'
    ■ Using image registry.k8s.io/ingress-nginx/controller:v1.9.4
[+] The 'ingress' addon is enabled

student@ubuntu:~$ kubectl get svc
NAME                TYPE        CLUSTER-IP    EXTERNAL-IP  PORT(S)    AGE
apache-service      ClusterIP   10.101.22.44   <none>       80/TCP     30s
kubernetes           ClusterIP   10.96.0.1      <none>       443/TCP    20m
nginx-service        ClusterIP   10.99.188.102  <none>       80/TCP     22s
```

Screenshot: Ingress addon enabled + both ClusterIP services running

Step 4 – Create the Ingress Rule

```
apiVersion: networking.k8s.io/v1
kind: Ingress
```



```

metadata:
  name: app-ingress
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  rules:
  - http:
      paths:
      - path: /apache
        pathType: Prefix
        backend:
          service:
            name: apache-service
            port:
              number: 80
      - path: /nginx
        pathType: Prefix
        backend:
          service:
            name: nginx-service
            port:
              number: 80

```

```

kubectl apply -f ingress.yaml
kubectl get ingress

```

```

student@ubuntu: ~
student@ubuntu:~$ kubectl apply -f ingress.yaml
ingress.networking.k8s.io/app-ingress created

student@ubuntu:~$ kubectl get ingress
NAME      CLASS  HOSTS  ADDRESS  PORTS  AGE
app-ingress  nginx  *      192.168.49.2  80     45s

student@ubuntu:~$ curl http://192.168.49.2/nginx | head -5
% Total    % Received % Xferd  Avg Speed   Time    Time     Time  Current
           Dload  Upload   Total   Spent    Left  Speed
100  615  100  615    0     0  120k      0  --:--:-- --:--:-- --:--:--  120k
<!DOCTYPE html>
<html>
<head><title>Welcome to nginx!</title></head>
student@ubuntu:~$ curl http://192.168.49.2/apache | grep title
<title>Apache2 Ubuntu Default Page: It works</title>

```

Screenshot: Ingress routing working – /nginx returns NGINX page, /apache returns Apache page

The ADDRESS field was blank for about 2 minutes after applying ingress.yaml – I got a bit scared but then it assigned 192.168.49.2 automatically. Also the rewrite-target annotation is important otherwise it returns 404.

☒ All 5 Kubernetes Assignments Completed